ObjectIR

Version 2

Language Specification

A stack-based intermediate representation
and virtual machine definition

Source format:	TextIR (human-readable), JSON, FOB/IR v3 (binary)
Execution model:	Stack machine, one evaluation stack per call frame
Type model:	Untyped stack; lazy coercion at instruction boundaries
Revision:	March 21, 2026

This document is the normative specification of the ObjectIR language: its source format, module structure, type system, and instruction set. Implementation-specific notes are confined to appendices.

Contents

1 Introduction **3**
1.1 Design Goals . . . 3
1.2 Document Conventions . . . 3

2 Module Structure **3**
2.1 Module Contents . . . 3
2.2 Module Identity . . . 4
2.3 Active Module . . . 4

3 Source Formats **4**

4 TextIR Syntax **4**
4.1 Module Header . . . 4
4.2 Type Declarations . . . 4
4.3 Member Declarations . . . 5
4.4 Grammar Summary . . . 5
4.5 Attributes (Annotations) . . . 6
4.6 String Escape Sequences . . . 6

5 Type System **6**
5.1 Primitive Types . . . 6
5.2 User-Defined Types . . . 7
5.3 Inheritance . . . 7
5.4 Null . . . 7
5.5 Value Coercion Semantics . . . 7

6 Instruction Set Reference **8**
6.1 Stack Manipulation . . . 8
6.2 Load Constants . . . 8
6.3 Arguments and Local Variables . . . 8
6.4 Field Access . . . 8
6.5 Arithmetic . . . 9
6.6 Logical . . . 9
6.7 Comparison . . . 9
6.8 Type Conversion and Testing . . . 10
6.9 Object and Array Creation . . . 10
6.10 Method Calls . . . 10
6.10.1 Call Resolution Order . . . 10
6.11 Control Flow . . . 11
6.11.1 Condition Kinds . . . 11

7 Exception Model **11**
7.1 Raising Exceptions . . . 11
7.2 Structured Exception Handling . . . 11
7.3 Exception Propagation . . . 12
7.4 Control-Flow Exception Tokens . . . 12

8 Module Schema (JSON / In-Memory) **12**
8.1 Top-Level Object . . . 12
8.2 Type Object . . . 12
8.3 Field Object . . . 12

8.4 Method Object . . . 13
8.5 Instruction Object . . . 13
8.6 Condition Object . . . 13
8.7 Attribute Object . . . 14

9 FOB/IR v3 Binary Format **14**
9.1 File Layout . . . 14
9.2 Binary Opcode Map . . . 14

10 Host Interoperability **15**
10.1 Host Type Registration . . . 15
10.2 Native Method Bindings . . . 15
10.3 Observability Hooks . . . 15

11 Runtime Conformance Requirements **16**

12 Worked Examples **16**
12.1 Hello World . . . 16
12.2 Fibonacci (Recursive) . . . 16
12.3 Instance Classes and Fields . . . 17
12.4 Exception Handling . . . 18

13 Known Limitations (v2) **19**

A Recommended Built-in Native Method Bindings **19**

B Opcode Quick Reference **20**

Introduction

ObjectIR is a typed, object-oriented intermediate representation (IR) designed to be simple to parse, simple to execute, and straightforward to target from higher-level languages or toolchains.

A program is expressed as one or more *modules*, each containing class and interface definitions whose methods are composed of explicit stack-machine instructions. The IR sits at the same conceptual level as JVM bytecode or CIL, but its primary surface format (*TextIR*) is human-readable and hand-writable.

Design Goals

- **Human-readable source.** TextIR is the canonical format: a whitespace-significant, keyword-driven text form that lowers to the same module representation as the JSON and binary forms.
- **Multiple input formats.** A module may be delivered as TextIR source, a JSON object graph, or a compact FOB/IR v3 binary. All three forms are semantically equivalent.
- **Embeddable.** A conforming runtime must allow host code to load a module, invoke individual methods, and register external (native) type and method bindings without touching a parser.
- **Lazy coercion.** Values on the evaluation stack carry no static type tag. Numeric widening and other conversions are performed at instruction boundaries according to well-defined rules.
- **Structured control flow.** Loops, conditionals, and exception handlers are first-class instructions, not jump targets.

Document Conventions

- **Normative** text uses **shall** / **must**. **Informative** text uses *should* / *may*.
- Grammar non-terminals are written ⟨*like-this*⟩; terminals are written `like-this`.
- Instruction mnemonics are typeset **`like-this`**.
- Type names are typeset `like-this`.

Module Structure

A *module* is the top-level compilation and loading unit of ObjectIR.

Module Contents

A well-formed module contains:

- A **name**: a non-empty identifier string.
- A **version**: a three-part numeric tuple (*major.minor.patch*), where *patch* is optional and defaults to `0`.
- An ordered list of **type definitions**: classes, interfaces, structs, and enums.
- A (reserved) **free-function table**, intended for future module-level functions outside any type. Runtimes must accept and ignore this field if present.

Module Identity

Two modules are considered the *same module* if and only if their names are identical. Version numbers are informational; the runtime does not perform version enforcement unless the host explicitly requires it.

Active Module

A runtime instance holds **exactly one** active module at a time. Loading a new module replaces the previous one. A *full reset* also clears all static field state; a *module-only reload* replaces the type table while optionally preserving heap state.

Source Formats

ObjectIR defines three equivalent surface representations of a module. All three must express identical semantics; a conforming tool or runtime may accept any subset.

Format	Description
TextIR	Human-readable source text (see Section 4). The canonical authoring format. A conforming parser produces a module representation equivalent to the JSON form.
JSON	A JSON object whose structure mirrors the module schema (Section 8). Property names are camelCase. This is the recommended interchange format between tools.
FOB/IR v3	A compact binary encoding of the module schema (Section 9). Intended for distribution and fast loading.

Auto-detection hint: A string whose first non-whitespace character is `{` or `[` should be treated as JSON; all other strings should be treated as TextIR. Implementations may expose this heuristic as an `Auto` format option.

TextIR Syntax

TextIR is a line-oriented, brace-delimited language. Whitespace (outside string literals) is insignificant except as a token separator. Comments begin with `//` and extend to the end of the line.

Module Header

Every TextIR file must begin with a module header:

```
module <name> version <major>.<minor>[.<patch>]
```

Type Declarations

Keyword	Kind	Notes
`class`	Reference type	Instances are heap-allocated objects.
`interface`	Contract type	Declares virtual methods; no fields.
`struct`	Value type	Semantics are host-defined.
`enum`	Enumeration type	Members are named integer constants.

Any combination of the following *access modifiers* may precede the type keyword: `public`, `private`, `protected`, `internal`.

The following *type modifiers* may also precede the keyword: `abstract`, `sealed`.

Inheritance and interface implementation are expressed with `::`

```
class Animal {
    field name: string
    virtual method Speak() -> void { ret }
}

class Dog : Animal {
    override method Speak() -> void implements Animal.Speak {
        ldstr "Woof!"
        call System.Console.WriteLine(string) -> void
        ret
    }
}
```

Member Declarations

```
// Field
[static] field <name>: <type>

// Method
[static] [virtual] [override] [abstract]
method <name>(<params>) -> <rettype> [implements <Interface>.<
    MethodName>]
{
    [local <name>: <type>]*
    <instructions>
}

// Constructor
constructor(<params>) {
    [local <name>: <type>]*
    <instructions>
}
```

Parameter lists and local variable declarations share the syntax `name: type`, comma-separated.

Grammar Summary

```
module-decl   ::= "module" name "version" version-num
version-num   ::= int "." int ["." int]

type-decl     ::= access-mod* type-mod* type-kw name [":" base-name]
                  "{" member* "}"
type-kw       ::= "class" | "interface" | "struct" | "enum"
access-mod    ::= "public" | "private" | "protected" | "internal"
type-mod      ::= "abstract" | "sealed"

member        ::= field-decl | method-decl | ctor-decl
field-decl    ::= ["static"] "field" name ":" type-name
method-decl   ::= method-mod* "method" name "(" params ")" "->" type-name
                  ["implements" qualified-name] "{" body "}"
method-mod    ::= "static" | "virtual" | "override" | "abstract"
ctor-decl     ::= "constructor" "(" params ")" "{" body "}"
```

```
params        ::= (name ":" type-name (","  name ":" type-name)*)?
body          ::= ("local" name ":" type-name)* instruction*
instruction   ::= opcode [operand]
```

Attributes (Annotations)

Custom attributes may be applied to types, fields, and methods by placing `@AttributeName` (with an optional positional argument list) immediately before the declaration:

```
@Serializable
class PlayerData {
    @NonSerialized
    field cachedHash: int32

    @Deprecated("Use SaveAsync instead")
    method Save() -> void { ret }
}
```

A runtime must attach attribute metadata to the corresponding declaration; how it is stored and queried is implementation-defined.

String Escape Sequences

Within string literals the following escape sequences are defined:

Sequence	Meaning
`\n`	Line feed (U+000A)
`\t`	Horizontal tab (U+0009)
`\r`	Carriage return (U+000D)
`\\`	Backslash (U+005C)
`\"`	Double quote (U+0022)

Type System

Primitive Types

ObjectIR defines a fixed set of primitive types. Type names in operands are **case-insensitive**: the runtime normalises them to lower-case before lookup, so `System.String`, `system.string`, and `string` are all equivalent.

Primary IR name(s)	Alias(es)	Width / Semantics
`void`	`system.void`	No value; return type only.
`bool`	`system.boolean`	Boolean true/false.
`int8`	`system.sbyte`	8-bit signed integer.
`uint8`	`system.byte`	8-bit unsigned integer.
`int16`	`system.int16`	16-bit signed integer.
`uint16`	`system.uint16`	16-bit unsigned integer.
`int32`	`system.int32`	32-bit signed integer.
`uint32`	`system.uint32`	32-bit unsigned integer.
`int64`	`system.int64`	64-bit signed integer.
`uint64`	`system.uint64`	64-bit unsigned integer.
`float32`, `single`	`system.single`	32-bit IEEE 754 float.

Primary IR name(s)	Alias(es)	Width / Semantics
`float64`, `double`	`system.double`	64-bit IEEE 754 float.
`char`	`system.char`	Unicode scalar (UTF-16 unit).
`string`	`system.string`	Immutable Unicode character sequence.
`object`	`system.object`	Top type; any value.
`decimal`	`system.decimal`	High-precision fixed-point decimal.
`datetime`	`system.datetime`	Date and time instant.
`timespan`	`system.timespan`	Duration / time interval.
`guid`	`system.guid`	128-bit globally unique identifier.

User-Defined Types

A type defined in the module is either a **class**, **interface**, **struct**, or **enum**. At runtime, instances of user-defined types are *managed objects*: heap-allocated records that carry:

- A **type name** identifying the declared type.
- A **field store**: a mapping from field name to value.
- An optional **host backing object** supplied by the host runtime (see Section 10).
- Zero or more **attribute instances** attached at load time.

Inheritance

A class may declare at most one *base class*. A class or struct may declare any number of *implemented interfaces*. Runtimes are not required to perform inheritance-aware type checks in v2 (see Section 13).

Null

`null` is a valid value for any non-primitive type. The **`ldnull`** instruction places it on the stack.

Value Coercion Semantics

Because the evaluation stack carries untyped values, the runtime applies the following coercion rules at instruction boundaries:

Target type	Rule
`bool`	`null`, integer `0`, and empty string `""` are `false`; all other values are `true`.
integer type	Floating-point values are truncated toward zero. Strings are parsed as base-10 integers using the invariant locale. Out-of-range results are implementation-defined but must not raise an unhandled exception.
floating-point	Integer values are widened exactly. Strings are parsed as decimal floating-point using the invariant locale.
`string`	Any non-null value is converted to its canonical string representation. `null` converts to `null`.
`object`	Identity; any value is valid.

Arithmetic instructions select *float mode* (operands widened to 64-bit float) when either operand is a floating-point value or a string; otherwise *integer mode* (64-bit signed) is used.

Instruction Set Reference

ObjectIR uses a **stack machine** execution model. Each method invocation creates a new *call frame* that owns:

- An **evaluation stack** of untyped values.
- A named **argument table** (index `0` / name `this` is the receiver for instance methods).
- A named **local variable table**.
- A **pending exception** slot.
- A **program counter** (index into the instruction list).

Operands are described using the JSON operand format of Section 8.

Stack Manipulation

Opcode	Description
`nop`	No-operation; advances the program counter only.
`dup`	Duplicate the top value on the evaluation stack.
`pop`	Discard the top value.
`ldnull`	Push `null` onto the stack.

Load Constants

Opcode	Description
`ldc`	Push a typed constant. Operand: `{value: ..., type: "<typename>"}`. `type` defaults to `int64`.
`ldc.i4`	Push a 32-bit integer constant. Operand: bare integer.
`ldc.i8`	Push a 64-bit integer constant. Operand: bare integer.
`ldc.r4`	Push a 32-bit float constant. Operand: bare number.
`ldc.r8`	Push a 64-bit float constant. Operand: bare number.
`ldstr`	Push a string constant. Operand: `{value: "..."}` or a bare quoted string.

Arguments and Local Variables

Opcode	Description
`ldarg`	Push a method argument. Operand: parameter name or 0-based index. Index `0` / name `this` is the receiver.
`starg`	Pop a value and store it in the named argument slot.
`ldloc`	Push a local variable by name.
`stloc`	Pop a value and store it in the named local variable.

Field Access

Opcode	Description
`ldfld`	Pop an object (or use the implicit receiver) and push the value of the named instance field. If the object has a host backing, the runtime *should* read the corresponding property on the host object before consulting the field store.
`stfld`	Pop a value, then pop an object; store the value in the named instance field (and on the host backing, if present).
`ldsfld`	Push the value of a static field. Operand: `{field: {declaringType, name}}`.
`stsfld`	Pop a value and store it in the named static field.

Arithmetic

Opcode	Description
`add`	Pop b, pop a, push $a + b$. In float mode, also performs string concatenation when either operand is a string.
`sub`	Pop b, pop a, push $a - b$.
`mul`	Pop b, pop a, push $a \times b$.
`div`	Pop b, pop a, push $a \div b$.
`rem`	Pop b, pop a, push $a \bmod b$.
`neg`	Pop v, push $-v$.

Mode selection follows Section 5.5: if either operand is floating-point (or a string for `add`), both are promoted to 64-bit float; otherwise 64-bit signed integer arithmetic is used.

Logical

Opcode	Description
`not`	Pop v, push the boolean negation: `!ToBool(`v`)`.

Comparison

Each opcode pops b then a and pushes a `bool`.

Opcode	Condition
`ceq`	$a = b$
`cne`	$a \neq b$
`cgt`	$a > b$
`cge`	$a \geq b$
`clt`	$a < b$
`cle`	$a \leq b$

String comparisons use ordinal (byte-by-byte) ordering. Boolean values are supported for `ceq` and `cne` only. All other operand pairs are promoted to 64-bit float before comparison.

Type Conversion and Testing

Opcode	Description
`conv`	Pop a value and push it converted to the named target type according to the coercion rules in Section 5.5.
`castclass`	Assert that the top-of-stack managed object has the given type name; raise a cast error if it does not. (Exact type-name match only in v2; see Section 13.)
`isinst`	Pop a value; push `true` if it is a managed object with the given type name, otherwise `false`.

v2 limitation: `castclass` and `isinst` perform *exact* type-name matching. Walking an inheritance chain is not required of conforming implementations in this version.

Object and Array Creation

Opcode	Description
`newobj`	Create a new managed object of the named type and push it. If the type is a registered host type, the runtime *should* also construct the corresponding host backing object.
`newarr`	Push a new, empty, dynamically-typed array.
`ldelem`	Pop an index (0-based integer), pop an array; push `array[index]`.
`stelem`	Pop a value, pop an index, pop an array; set `array[index] = value`.

Arrays in ObjectIR v2 are untyped and variable-length. Typed array variants (e.g. `int[]`) are not defined in this version.

Method Calls

Opcode	Description
`call`	Pop arguments left-to-right, then pop the receiver (for instance methods); invoke the target method. Operand: `{method: {declaringType, name, returnType, parameterTypes[]}}`.
`callvirt`	Same as `call`, but dispatch is virtual: the actual type of the receiver determines which override is executed.
`ret`	Return from the current frame. If a value is present on the evaluation stack it is transferred to the caller's stack.

Call Resolution Order

A conforming runtime must attempt the following sequence for every `call` and `callvirt`:

1. **Exact native/host lookup.** Match the full signature string `DeclaringType.Name(param0,param1,...)` against the registered host method table.
2. **Simplified native/host lookup.** Retry with parameter type names reduced to their unqualified, lower-case short names (e.g. `System.String` → `string`).

3. **Host reflection.** If the runtime supports reflective dispatch and it is enabled, invoke the method on the host backing object.
4. **IR method dispatch.** Look up the method in the loaded module's type table.

If none of the steps resolve the call, a **missing-method** error is raised.

Control Flow

Opcode	Description
`if`	Evaluate the condition operand (Section 6.11.1). If truthy, execute `thenBlock`; otherwise execute the optional `elseBlock`.
`while`	Repeatedly evaluate the condition and execute `body`. A **`break`** inside `body` exits the loop; a **`continue`** skips to the next condition evaluation.
`break`	Exit the nearest enclosing **`while`**.
`continue`	Skip to the next iteration of the nearest **`while`**.
`try`	Execute `tryBlock`. If an exception is raised, test each `catchBlock` in order (Section 7). Execute the optional `finallyBlock` in all cases.
`throw`	Pop a value and raise it as a pending exception on the current frame.

Condition Kinds

A condition embedded in an **`if`** or **`while`** operand has a `kind` field that determines how it is evaluated:

Kind	Evaluation
`stack`	Pop the top-of-stack value and coerce it to `bool` (Section 5.5). An empty stack is treated as `false`.
`binary`	Pop r, pop l; apply the named comparison opcode (**`ceq`**, **`cgt`**, ...) and use the result.
`expression`	Execute a single embedded instruction and pop the result.
`block`	Execute a sequence of instructions and pop the final result.

Exception Model

Raising Exceptions

The **`throw`** instruction accepts any value: a managed object, a host exception type, or a plain string. The value is stored in the *pending exception* slot of the current frame and instruction execution in that frame halts.

Structured Exception Handling

The **`try`** instruction provides first-class structured exception handling. Its operand defines up to three regions:

- `tryBlock` — the guarded instructions.
- `catchBlocks` — an ordered list of catch clauses, each with an optional `type` field and a `body`.

- `finallyBlock` — instructions executed unconditionally after the try/catch, regardless of outcome.

Catch clauses are matched in declaration order against the pending exception:

1. **Catch-all** (no `type` field) — matches any value.
2. **Type name match** — the pending exception is a managed object whose type name equals the catch clause's `type`.
3. **Host type match** — the host runtime may additionally apply assignability checks against registered host types.

Exception Propagation

If no catch clause in the current frame matches, the pending exception propagates to the caller's frame. This continues until either a matching clause is found or the call stack is exhausted. An unhandled exception at the top of the stack is delivered to the host via the exception hook.

Control-Flow Exception Tokens

The **`break`** and **`continue`** instructions are implemented as *structured control-flow tokens*. They must not be catchable by user **`try`**/**`catch`** blocks and must not propagate outside the enclosing **`while`** instruction.

Module Schema (JSON / In-Memory)

This section defines the canonical JSON structure of a module. The same structure is used as the in-memory representation produced by a TextIR parser and consumed by a runtime. All property names are camelCase.

Top-Level Object

```
{
  "name":      "<module-name>",
  "version":   "<major>.<minor>.<patch>",
  "types":     [ <TypeObject>, ... ],
  "functions": null   // reserved; must be accepted and ignored
}
```

Type Object

```
{
  "kind":            "class" | "interface" | "struct" | "enum",
  "name":            "<SimpleName>",
  "_namespace":      "<Namespace>" | null,
  "baseType":        "<QualifiedName>" | null,
  "baseInterfaces": [ "<QualifiedName>", ... ],
  "fields":          [ <FieldObject>, ... ],
  "methods":         [ <MethodObject>, ... ]
}
```

Field Object

```
{
  "name":       "<name>",
  "type":       "<typename>",
```

```
  "isStatic":   true | false,
  "attributes": [ <AttributeObject>, ... ]
}
```

Method Object

```
{
  "name":          "<name>",
  "returnType":    "<typename>",
  "isStatic":      true | false,
  "isVirtual":     true | false,
  "isOverride":    true | false,
  "isAbstract":    true | false,
  "parameters":    [ { "name": "...", "type": "..." }, ... ],
  "locals":        [ { "name": "...", "type": "..." }, ... ],
  "instructions": [ <InstructionObject>, ... ],
  "attributes":    [ <AttributeObject>, ... ]
}
```

Instruction Object

```
{
  "opCode":  "<mnemonic>",
  "operand": <any JSON value>
}
```

The `operand` is opcode-specific. Common patterns:

Opcode(s)	Operand form
ldstr	Bare string or `{"value":"..."}`
ldc, **ldc.***	`{"value": ..., "type": "<typename>"}` or bare number
ldarg, **starg**, **ldloc**, **stloc**	Parameter/local name (string) or 0-based index (integer)
ldfld, **stfld**	Field name string
ldsfld, **stsfld**	`{"field":{"declaringType":"...","name":"..."}}`
call, **callvirt**	`{"method":{"declaringType":"...","name":"...", "returnType":"...","parameterTypes":["..."]}}`
newobj, **conv**, **castclass**, **isinst**	Type name string
if	`{"condition":<CondObj>,"thenBlock":[...], "elseBlock":[...]}`
while	`{"condition":<CondObj>,"body":[...]}`
try	`{"tryBlock":[...],"catchBlocks":[...], "finallyBlock":[...]}`

Condition Object

```
{
  "kind":      "stack" | "binary" | "expression" | "block",
  "operation": "<opcode>",              // for "binary"
  "body":      [ <InstructionObject>, ... ]  // for "expression" / "block
     "
}
```

Attribute Object

```
{
  "name": "<AttributeTypeName>",
  "args": [ <value>, ... ]
}
```

FOB/IR v3 Binary Format

The FOB/IR v3 format is a compact binary encoding of the module schema defined in Section 8. It is intended for distribution and fast loading; the semantics are identical to the JSON and TextIR forms.

File Layout

Section	Contents
Header	4-byte magic `0x464F4249` (`FOBI`), 1-byte format version (`0x03`), module name (length-prefixed UTF-8), version triple (three `uint16`).
String pool	A length-prefixed table of interned UTF-8 strings referenced by index throughout the rest of the file.
Type table	Sequential type records: kind byte, name index, namespace index, base type index, interface index list, field count + field records, method count + method records.
Method bodies	Per-method: parameter count + parameter records, local count + local records, instruction count + instruction records.

Binary Opcode Map

Each instruction is stored as a 1-byte opcode followed by a variable-length operand.

Byte	Mnemonic	Byte	Mnemonic
0x00	nop	0x10	ldsfld
0x01	ldc	0x11	stsfld
0x02	ldstr	0x12	newobj
0x03	ldarg	0x13	newarr
0x04	starg	0x14	ldelem
0x05	ldloc	0x15	stelem
0x06	stloc	0x16	call
0x07	add	0x17	callvirt
0x08	sub	0x18	ret
0x09	mul	0x19	if
0x0A	div	0x1A	while
0x0B	rem	0x1B	break
0x0C	neg	0x1C	continue
0x0D	ceq	0x1D	try
0x0E	cne	0x1E	throw
0x0F	ldfld	0x1F	conv
0x20	castclass	0x21	isinst

Byte	Mnemonic	Byte	Mnemonic
0x22	dup	0x23	pop
0x24	ldnull	0x25	not
0x26	cgt	0x27	cge
0x28	clt	0x29	cle
0x2A	stfld	0x2B	ldc.i4
0x2C	ldc.i8	0x2D	ldc.r4
0x2E	ldc.r8		

Host Interoperability

This section defines requirements for runtimes that allow host code to expose native types and methods to IR programs. All features in this section are **optional**; a minimal runtime that only executes pure IR code need not implement them.

Host Type Registration

A runtime *may* allow the host to register *external types* under IR type names. Once registered:

- **`newobj`** for that type name constructs a host instance and attaches it as the backing of the managed object wrapper.
- **`ldfld`** / **`stfld`** consult the host object's property store before the managed object's field dictionary.
- **`ldsfld`** / **`stsfld`** may access static members on the host type.

Native Method Bindings

A runtime *may* allow the host to register *native method bindings*: callbacks keyed by a call signature string of the form:

```
DeclaringType.MethodName(param0Type,param1Type,...)
```

When the runtime resolves a **`call`** or **`callvirt`**, it checks the native binding table before performing IR method lookup (steps 1–3 of Section 6.10.1).

A native binding callback receives:

- The receiver object (or a null equivalent for static calls).
- A positional list of argument values in declaration order.

It returns a single result value (or a null equivalent for void methods).

Observability Hooks

A runtime *should* provide the following hooks for diagnostics and tooling:

- **Step hook** — invoked before each instruction with the current frame and instruction as arguments. Useful for debuggers, profilers, and instruction-level traces.
- **Exception hook** — invoked when an exception escapes the top of the call stack.

Runtime Conformance Requirements

A conforming ObjectIR v2 runtime **must**:

1. Accept at least one of the three module formats (TextIR, JSON, FOB/IR v3) as input.
2. Maintain a call stack of frames with independent evaluation stacks.
3. Implement all opcodes defined in Section 6.
4. Apply the coercion rules of Section 5.5 at instruction boundaries.
5. Implement the call resolution order of Section 6.10.1.
6. Implement structured exception handling as defined in Section 7, including the requirement that `break`/`continue` tokens are not user-catchable.
7. Expose an entry-point invocation mechanism (canonically targeting a static `Program.Main` with no parameters).
8. Expose a named method invocation mechanism allowing the host to call any method by qualified name with arbitrary arguments.
9. Expose a named method existence check that inspects module metadata without executing code.

A conforming runtime *should* additionally:

10. Implement host type registration (Section 10).
11. Implement native method binding (Section 10).
12. Implement step and exception observability hooks.
13. Pre-register the built-in `System.Console` and `System.String` native bindings listed in Appendix A.

Worked Examples

Hello World

Listing 1: Hello World in TextIR

```
module HelloWorld version 1.0.0

class Program {
    static method Main() -> void {
        ldstr "Hello, ObjectIR!"
        call System.Console.WriteLine(string) -> void
        ret
    }
}
```

Fibonacci (Recursive)

Listing 2: Recursive Fibonacci

```
module Fibonacci version 1.0.0

class Program {
    static method Fib(n: int32) -> int32 {
        ldarg n
```

```
        ldc.i4 2
        clt
        if { "kind": "stack",
             "thenBlock": [
               { "opCode": "ldarg", "operand": "n" },
               { "opCode": "ret" }
             ] }

        ldarg n
        ldc.i4 1
        sub
        call Program.Fib(int32) -> int32

        ldarg n
        ldc.i4 2
        sub
        call Program.Fib(int32) -> int32

        add
        ret
    }

    static method Main() -> void {
        ldc.i4 10
        call Program.Fib(int32) -> int32
        call System.Console.WriteLine(int32) -> void
        ret
    }
}
```

Instance Classes and Fields

Listing 3: Instance class with constructor

```
module ClassDemo version 1.0.0

class Vector2D {
    field x: float32
    field y: float32

    constructor(x: float32, y: float32) {
        ldarg this
        ldarg x
        stfld x
        ldarg this
        ldarg y
        stfld y
        ret
    }

    // returns sqrt(x*x + y*y)
    method Length() -> float32 {
        ldarg this
        ldfld x
        ldarg this
        ldfld x
        mul
        ldarg this
        ldfld y
```

```
        ldarg this
        ldfld y
        mul
        add
        call System.Math.Sqrt(float64) -> float64
        conv float32
        ret
    }
}

class Program {
    static method Main() -> void {
        ldc.r4 3.0
        ldc.r4 4.0
        newobj Vector2D
        callvirt Vector2D.Length() -> float32
        call System.Console.WriteLine(object) -> void
        ret
    }
}
```

Exception Handling

Listing 4: try / catch / finally

```
module ExceptionDemo version 1.0.0

class MyError {
    field message: string
    constructor(message: string) {
        ldarg this
        ldarg message
        stfld message
        ret
    }
}

class Program {
    static method Main() -> void {
        try {
            "tryBlock": [
                { "opCode": "newobj",  "operand": "MyError" },
                { "opCode": "throw" }
            ],
            "catchBlocks": [ {
                "type": "MyError",
                "variable": "err",
                "body": [
                    { "opCode": "ldstr", "operand": "caught!" },
                    { "opCode": "call",
                      "operand": { "method": {
                        "declaringType": "System.Console",
                        "name": "WriteLine",
                        "returnType": "void",
                        "parameterTypes": ["string"] } } }
                ]
            } ],
            "finallyBlock": [
                { "opCode": "ldstr", "operand": "always runs" },
```

```
                  { "opCode": "call",
                    "operand": { "method": {
                      "declaringType": "System.Console",
                      "name": "WriteLine",
                      "returnType": "void",
                      "parameterTypes": ["string"] } } }
              ]
          }
          ret
      }
}
```

Known Limitations (v2)

The following features are explicitly **out of scope** for ObjectIR v2 and are reserved for future versions:

- **Generics.** Generic type parameters and instantiation are not defined. Generic host APIs must be exposed through native method bindings.
- **Typed arrays.** All arrays are untyped. Distinct `int[]`, `float[]`, etc. are not distinguished.
- **Inheritance-aware type tests.** `castclass` and `isinst` match by exact type name only.
- **Method overloading on IR-defined types.** Only one method with a given name is permitted per IR type in v2; overloading resolution applies only to host native bindings.
- **Module-level free functions.** The `functions` field is reserved; runtimes must accept it but need not execute it.
- **Concurrent execution.** The specification does not define thread-safety semantics for concurrent use of a single runtime instance.

Recommended Built-in Native Method Bindings

A conforming runtime *should* pre-register the following native bindings. The table describes the expected observable behaviour; the implementation mechanism is host-defined.

Signature	Expected behaviour
`System.Console.WriteLine(string)`	Write the argument followed by a newline to standard output.
`System.Console.WriteLine(int32)`	Same, with an integer argument.
`System.Console.WriteLine(object)`	Same, converting the argument to string first.
`System.Console.WriteLine(string,string)`	Concatenate the two arguments and write a line.
`System.Console.WriteLine(string,object)`	Format the first argument as a template with the second; write a line.
`System.Console.Write(string)`	Write the argument to standard output without a newline.
`System.Console.Write(string,string)`	Concatenate the two arguments and write without a newline.
`System.Console.ReadLine()`	Read one line from standard input and return it as a string.
`System.Console.Clear()`	Clear the standard output terminal if supported; otherwise no-op.

Signature	Expected behaviour
`System.String.Concat(string,string)`	Return the concatenation of the two string arguments.
`System.String.Format(string,object)`	Return the first argument as a format template applied to the second.
`System.String.Format(string,object,object)`	Same with two substitution arguments.
`System.String.IsNullOrEmpty(string)`	Return `true` if the argument is `null` or `""`.
`System.Object.ToString()`	Return the string representation of the receiver.
`System.Object.GetType()`	Return a description of the receiver's runtime type.

Opcode Quick Reference

Mnemonic	Category	Stack effect
`nop`	Stack	—
`dup`	Stack	$v \to v,\ v$
`pop`	Stack	$v \to$
`ldnull`	Stack	$\to$ `null`
`ldc`	Constants	$\to c$
`ldc.i4`	Constants	$\to c_{\mathrm{i32}}$
`ldc.i8`	Constants	$\to c_{\mathrm{i64}}$
`ldc.r4`	Constants	$\to c_{\mathrm{f32}}$
`ldc.r8`	Constants	$\to c_{\mathrm{f64}}$
`ldstr`	Constants	$\to s$
`ldarg`	Variables	$\to a$
`starg`	Variables	$v \to$
`ldloc`	Variables	$\to l$
`stloc`	Variables	$v \to$
`ldfld`	Fields	$obj \to v$
`stfld`	Fields	$obj, v \to$
`ldsfld`	Fields	$\to v$
`stsfld`	Fields	$v \to$
`add`	Arithmetic	$a, b \to a{+}b$
`sub`	Arithmetic	$a, b \to a{-}b$
`mul`	Arithmetic	$a, b \to a{\times}b$
`div`	Arithmetic	$a, b \to a{\div}b$
`rem`	Arithmetic	$a, b \to a \bmod b$
`neg`	Arithmetic	$v \to -v$
`not`	Logical	$v \to \neg v$
`ceq`	Comparison	$a, b \to a{=}b$
`cne`	Comparison	$a, b \to a{\neq}b$
`cgt`	Comparison	$a, b \to a{>}b$
`cge`	Comparison	$a, b \to a{\geq}b$
`clt`	Comparison	$a, b \to a{<}b$
`cle`	Comparison	$a, b \to a{\leq}b$
`conv`	Type	$v \to v'$
`castclass`	Type	$v \to v$ (or error)
`isinst`	Type	$v \to$ `bool`

Mnemonic	**Category**	**Stack effect**
`newobj`	Objects	$\rightarrow obj$
`newarr`	Objects	$\rightarrow arr$
`ldelem`	Objects	$arr, i \rightarrow v$
`stelem`	Objects	$arr, i, v \rightarrow$
`call`	Calls	$args \ldots \rightarrow [r]$
`callvirt`	Calls	$args \ldots, obj \rightarrow [r]$
`ret`	Calls	$[v] \rightarrow$ (returns to caller)
`if`	Control	(condition-dependent)
`while`	Control	(condition-dependent)
`break`	Control	—
`continue`	Control	—
`try`	Control	—
`throw`	Control	$v \rightarrow$ (raises exception)